

CloudCLI Fork

Master Feature Product Requirements Document

Version 1.0 · **Date** 2026-08-06

Status Approved for phased multi-agent implementation

Audience Implementation agents and human reviewers

Companion Markdown (preferred for agents): `docs/prd/CloudCLI-Feature-PRD-v1.md`

This PRD specifies the full priority-ordered feature program: Isolated Workspaces, Canonical Run Spine, Interrupt Queue, Secrets Vault, One Work System + PRD Delivery Graph, Ship Loop, Context Pack Compiler, Automation Kernel, Failover Playbooks, Workspace Capsule, and Phase-2 platforms. Each phase includes goals, data models, APIs, UI, tests, acceptance criteria, and multi-agent parallelization guidance.

Critical path: P0 → P1 ■ P2 → P3 → P5/P6/P7 → P8 → P9 → P10 → P11

Table of Contents

- 0. How to use with parallel agents
- 1. Vision & principles
- 2. Current state (code anchors)
- 3. Priority roadmap
- 4. Phase 0 — Shared contracts
- 5. Phase 1 — Isolated Agent Workspaces
- 6. Phase 2 — Canonical Run Spine + Observatory
- 7. Phase 3 — Interrupt Queue
- 8. Phase 4 — Secrets Hygiene
- 9. Phase 5 — One Work System + PRD Graph
- 10. Phase 6 — Ship Loop
- 11. Phase 7 — Context Pack Compiler
- 12. Phase 8 — Automation Kernel
- 13. Phase 9 — Failover Playbooks
- 14. Phase 10 — Workspace Capsule + Doctor
- 15. Phase 11 — Phase-2 platforms
- 16–19. Cross-cutting architecture, playbooks, metrics, open questions
- 20–23. Appendices & document history

0. How to use this document with parallel agents

0.1 Rules of engagement

- **Implement in phase order for foundations** (P1 → P2 → P3...). Later phases may start design/stubs only after their “Depends on” checklist is green.
- **Do not invent provider capabilities.** Inventory comes from real files / real CLI sources (project memory constraint).
- **All temp work** lives under tmp/cloudcli/<task-id>/ only.
- **No silent cross-provider pollution** for MCP/skills — define in CloudCLI, enable per agent.
- Prefer **provider-native configs** so CLIs still work outside CloudCLI.
- Every PR/slice must include: schema/migration (if any), service, routes, tests, minimal UI hook, and a short “agent handoff” note in the PR body.

0.2 Parallelization matrix (who can run concurrently)

Workstream ID	Slice	Can start after	Parallel with	Owns paths (primary)
WS-P1A	Worktree service + git ops	now	WS-P1B design	server/modules/workspaces/ (new)
WS-P1B	Kanban/MC/webhook integration	WS-P1A API stable	WS-P1C	server/modules/kanban/, mission-control/, webhooks/
WS-P1C	Worktree UI (diff, merge, discard)	WS-P1A list/diff APIs	WS-P2A design	src/components/workspaces/ (new)
WS-P2A	agent_runs schema + repository	now (schema only; no consumers yet)	WS-P1A	server/modules/database/, server/modules/runs/ (new)
WS-P2B	Event append + chat-run bridge	WS-P2A	WS-P1B	server/modules/websocket/, runs/
WS-P2C	Run Observatory UI	WS-P2B read APIs	WS-P3A	src/components/runs/ (new)
WS-P3A	Interrupt queue service	WS-P2A (link run_id) preferred	WS-P1C	server/modules/interrupt-queue/ (new)
WS-P3B	Interrupt queue UI + mobile	WS-P3A	WS-P4A	src/components/interrupt-queue/
WS-P4A	Secrets store backend	now	WS-P1A	server/modules/secrets/ (new)
WS-P4B	Credential + MCP env migration	WS-P4A	WS-P2B	schema, credentials routes, MCP catalog
WS-P5A	TaskMaster → Kanban importer	WS-P2A recommended	WS-P5B	server/modules/kanban/, taskmaster routes
WS-P5B	PRD → delivery graph	WS-P5A board model clear	WS-P6A	prd-editor, kanban-generate
WS-P6A	Ship Loop (PR/CI)	WS-P1A	WS-P5B	server/routes/git.js → server/modules/git/
WS-P7A	Context pack compiler	WS-P2A + memory APIs	WS-P6A	server/modules/context-pack/ (new)
WS-P8A	Automation kernel	WS-P2A	WS-P7A	server/modules/automation/ (new)

Workstream ID	Slice	Can start after	Parallel with	Owens paths (primary)
WS-P8B	Migrate MC/webhooks/kanban schedules	WS-P8A	WS-P9A	respective modules
WS-P9A	Failover playbooks	WS-P2A + auth-health	WS-P8B	server/modules/providers/
WS-P10A	Workspace capsule + doctor	WS-P4A + catalog	WS-P9A	server/modules/stack/(new)
WS-P11+	Phase-2 platforms	P1–P2 done	selective	see §12

0.3 Definition of Done (every phase)

- [] Unit tests for pure logic
- [] Integration tests for DB + at least one happy path + one failure path
- [] API types exported (TS) matching routes
- [] UI can exercise the happy path (or CLI/doctor if backend-only phase)
- [] Migration is additive and reverse-documented
- [] No plaintext secrets introduced
- [] Events/logs do not dump secrets or full env
- [] tmp/cloudcli/ only for scratch

1. Vision

Turn CloudCLI from a multi-provider **agent launcher** into a multi-provider **agent operations platform**: isolated parallel work, durable runs, human attention routing, safe secrets, one work system, ship loop, budgeted context, unified automation, failover, and reproducible environments.

1.1 Product principles

Principle	Implication
Isolation first	Concurrent agents never share a dirty worktree
One run identity	Chat / Kanban / MC / webhook executions share run_id
Attention is scarce	Surface only actionable interrupts; support snooze
Secret-by-reference	Never store or log raw tokens when refs exist
One work system	Kill dual TaskMaster + Kanban cognitive load
Budgeted context	Prefer smallest reliable pack over “search everything”
Kernel before studio	Shared automation engine before visual builders
Failover before ML router	Health + handoff policies before outcome-driven routing
Provider-native	Projected configs remain usable outside CloudCLI

1.2 Non-goals (v1 program)

- Full multi-tenant team RBAC / org accounts (single-user + API key model remains primary)
- Training custom models or building a model marketplace
- Replacing upstream CLIs (Claude/Codex/Cursor/Grok/Kimi/...) — CloudCLI orchestrates them
- Pixel-perfect redesign of entire app chrome (incremental surfaces only)
- Full Semantic Brain / Eval Lab / Provider SDK as day-one (Phase 2)

2. Current state (code anchors)

Use these as ground truth; re-verify before coding if main has moved.

Area	Location	Gap
Kanban branch	server/modules/kanban/git-branch.service.ts	git checkout -b in shared projectPath
Kanban queue concurrency	server/modules/kanban/kanban-queue.service.ts	Concurrent runs allowed; no workspace isolation
Kanban task model	server/modules/database/schema.ts kanban_tasks	Rich fields; feature_branch only
Kanban runs	kanban_runs table	Thin: status, timestamps, exit code; little telemetry
Chat runs	server/modules/websocket/services/chat-run-registry.service.ts	In-memory map; not durable
Credentials	user_credentials.credential_value	Plaintext
Webhook secrets	webhook_sources.secret	Plaintext HMAC secret field
PRD generate	src/components/prd-editor/view/GenerateTasksModal.tsx	Stub: "ask Claude manually"
TaskMaster	server/routes/taskmaster.js, src/components/task-master/	Parallel task system vs native Kanban
Conversation search	session-conversations-search.service.ts	Claude + Codex only; regex/literal
Handoff	session-handoff.service.ts	Exists; not auto-wired to failover
Auth/MCP health	auth-health/, mcp-health.service.ts	Health signals; not playbooks
Notifications	system_notifications	Passive inbox; limited actions
MCP catalog	catalog + fan-out	Config-centric, not full doctor
Plugins	server/routes/plugins.js	Git install + auto start; weak trust
Temp rule	project memory	Must use tmp/cloudcli/

3. Priority roadmap (build order)

Phase	Name	Effort	Depends on
P0	Shared contracts & conventions	S	—
P1	Isolated Agent Workspaces	L	P0
P2	Canonical Run Spine + Run Observatory (v1)	L	P0 (P1 links workspaces)
P3	Interrupt Queue (Attention Budget)	M	P2 preferred
P4	Secrets Hygiene (Vault v1)	M	P0
P5	One Work System + PRD→Delivery Graph	M–L	P2
P6	Ship Loop (Diff → Test → PR → CI)	M–L	P1
P7	Context Pack Compiler	M	P2 + memory
P8	Shared Automation Kernel (+ migrate callers)	L	P2
P9	Provider Failover Playbooks	M–L	P2 + auth-health + handoff
P10	Workspace Capsule + Doctor	M	P4 + MCP catalog
P11	Phase-2 platforms (evals, swarm, SDK, full trust, browser evidence, semantic search, outcome router)	L+	P1–P2 minimum

Critical path: P0 → P1 ■ P2 → P3 → P5/P6/P7 → P8 → P9 → P10 → P11

4. Phase 0 — Shared contracts & conventions

4.1 Goals

Establish shared TypeScript types, ID formats, event envelope, error codes, and module layout so parallel agents do not invent incompatible shapes.

4.2 ID formats

Entity	Format	Example
run_id	run_<ulid>	run_01J...
workspace_id	ws_<ulid>	ws_01J...
secret_id	sec_<ulid>	sec_01J...
interrupt_id	int_<ulid>	int_01J...
pack_id	pack_<ulid>	pack_01J...
recipe_id	rec_<ulid>	rec_01J...
playbook_id	pb_<ulid>	pb_01J...

Use ULID (or existing UUID if ULID not yet in tree — pick one in P0 and stick to it; prefer ULID for sortability).

4.3 Event envelope (all durable events)

```
// server/shared/run-events.ts (new)
export type RunEventEnvelope = {
  event_id: string;           // evt_<ulid>
  run_id: string;
  ts: string;                 // ISO-8601
  source: 'chat' | 'kanban' | 'mission_control' | 'webhook' | 'system' | 'ship' | 'automation';
  type: string;               // dotted: run.started, tool.call, permission.requested, ...
  severity?: 'debug' | 'info' | 'warn' | 'error';
  payload: Record<string, unknown>; // JSON-serializable; secrets redacted
  seq?: number;               // monotonic per run_id
};
```

4.4 Error codes

```
export type CloudErrorCode =
  | 'WORKSPACE_DIRTY_CONFLICT'
  | 'WORKSPACE_CREATE_FAILED'
  | 'WORKSPACE_NOT_FOUND'
  | 'RUN_NOT_FOUND'
  | 'RUN_ALREADY_TERMINAL'
  | 'SECRET_NOT_FOUND'
  | 'SECRET_RESOLVE_FAILED'
  | 'INTERRUPT_NOT_FOUND'
  | 'INTERRUPT_ALREADY_RESOLVED'
  | 'PACK_BUDGET_EXCEEDED'
  | 'PLAYBOOK_NO_CANDIDATE'
  | 'SHIP_PR_FAILED'
  | 'STACK_DOCTOR_FAILED'
  | 'AUTOMATION_CYCLE'
  | 'AUTOMATION_TIMEOUT';
```

4.5 Module layout convention

```
server/modules/<domain>/
  index.ts
  <domain>.types.ts
  <domain>.repository.ts
  <domain>.service.ts
  <domain>.routes.ts
  tests/
src/components/<domain>/
  api/
  hooks/
  types.ts
```

view/

4.6 WebSocket fan-out

Extend chat/project websocket with:

```
type SystemWsEvent =
| { kind: 'run_event'; run_id: string; event: RunEventEnvelope }
| { kind: 'run_updated'; run: AgentRunSummary }
| { kind: 'interrupt_created' | 'interrupt_updated'; interrupt: InterruptItem }
| { kind: 'workspace_updated'; workspace: WorkspaceSummary }
| { kind: 'secret_rotated'; secret_id: string } // never include value
| { kind: 'notification_created'; /* existing */ };
```

4.7 Acceptance criteria (P0)

- Shared types package importable by server modules without circular deps
- Documented event type registry in server/shared/run-event-types.md
- ESLint path aliases for new modules

5. Phase 1 — Isolated Agent Workspaces

5.1 Problem

Kanban (and potentially other automations) can run concurrent agents against one project directory. Branch creation uses git checkout in the shared tree (git-branch.service.ts), so agents overwrite each other's working tree.

5.2 Goals

- Every autonomous run that mutates the repo gets an isolated Git worktree (or explicit sandbox mode).
- Live status: branch, HEAD SHA, dirty files, ahead/behind.
- Human controls: **Merge**, **Discard**, **Open diff**, **Cleanup**.
- Interactive chat may opt-in to workspace mode; default chat may remain on main working tree unless user enables isolation.

5.3 Non-goals (P1)

- Full PR creation (P6)
- Conflict-resolution agents
- Nested worktrees of worktrees

5.4 Data model

```
CREATE TABLE IF NOT EXISTS agent_workspaces (
  workspace_id TEXT PRIMARY KEY NOT NULL,
  project_id TEXT NOT NULL,
  run_id TEXT, -- nullable until run spine; backfill later
  task_id TEXT, -- optional kanban task
  mode TEXT NOT NULL DEFAULT 'git_worktree', -- git_worktree | sandbox_copy
  root_path TEXT NOT NULL, -- absolute path under project or tmp/cloudcli
  base_branch TEXT NOT NULL,
  base_sha TEXT,
  feature_branch TEXT NOT NULL,
  head_sha TEXT,
  status TEXT NOT NULL DEFAULT 'active',
  -- active | merging | merged | discarded | error | orphan
  last_error TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  cleaned_at DATETIME
);
CREATE INDEX IF NOT EXISTS idx_agent_workspaces_project ON agent_workspaces(project_id, status);
CREATE INDEX IF NOT EXISTS idx_agent_workspaces_run ON agent_workspaces(run_id);
CREATE INDEX IF NOT EXISTS idx_agent_workspaces_task ON agent_workspaces(task_id);
```

Path policy:

- Preferred: <project>/`.cloudcli/worktrees`/`<workspace_id>`/ (gitignored)
- Fallback if project not writable: `tmp/cloudcli/worktrees/<project_id>/<workspace_id>/`
- Never create worktrees outside these roots
- Ensure `.cloudcli/worktrees/` in project `.gitignore` via doctor (P10) or create helper

5.5 Service API

```
// server/modules/workspaces/workspace.service.ts
interface WorkspaceService {
  create(input: {
    projectId: string;
    projectPath: string;
    baseBranch?: string;      // default: current branch or main/master
    branchName?: string;      // default: feat/<task-or-run-slug>
    taskId?: string;
    runId?: string;
    mode?: 'git_worktree' | 'sandbox_copy';
  }): Promise<AgentWorkspace>;

  get(workspaceId: string): AgentWorkspace | null;
  list(projectId: string, filter?: { status?: string[] }): AgentWorkspace[];

  refreshStatus(workspaceId: string): Promise<WorkspaceStatus>;
  // status includes: dirty files, ahead/behind, head_sha, conflicts?

  getDiff(workspaceId: string, opts?: { base?: 'merge-base' | 'base_sha' }): Promise<DiffResult>;
  // DiffResult: files[{path, status, patch?}], summary{additions, deletions}

  mergeToBase(workspaceId: string, opts?: {
    strategy?: 'ff-only' | 'merge' | 'squash';
    deleteAfter?: boolean;
  }): Promise<MergeResult>;

  discard(workspaceId: string, opts?: { deleteBranch?: boolean }): Promise<void>;
  cleanup(workspaceId: string): Promise<void>; // remove worktree dir + git worktree prune

  /** Resolve cwd for a provider session bound to this workspace */
  resolveCwd(workspaceId: string): string;
}
```

Git commands (worktree mode):

```
git fetch --all --prune # optional, best-effort
git rev-parse --verify <base>
git worktree add -b <feature_branch> <root_path> <base>
# on discard:
git worktree remove --force <root_path>
git branch -D <feature_branch> # if deleteBranch
# on merge (from main repo):
git -C <project> merge --no-ff <feature_branch>
```

Concurrency: Use a per-project mutex around worktree add/remove/merge (in-process lock + file lock under `.cloudcli/locks/`).

Replace ensureFeatureBranch checkout behavior:

- `kanban-runner.service.ts`: before start, `workspaceService.create(...)`, set session cwd to `resolveCwd`, store `workspace_id` on run/task.
- Do **not** call `git checkout` on the primary project path for concurrent implement runs.

5.6 HTTP routes

```
POST /api/projects/:projectId/workspaces
GET /api/projects/:projectId/workspaces
GET /api/workspaces/:workspaceId
GET /api/workspaces/:workspaceId/status
GET /api/workspaces/:workspaceId/diff
POST /api/workspaces/:workspaceId/merge
```


POST /api/workspaces/:workspaceId/discard
POST /api/workspaces/:workspaceId/cleanup

Auth: existing JWT / platform user middleware.

5.7 Provider session wiring

When spawning provider processes for Kanban/MC/webhook:

- Pass `cwd = workspace.root_path`
- Ensure MCP/skills still resolve (absolute paths in catalog)
- Record `workspace_id` on session metadata if available

Chat opt-in: composer toggle “Isolated workspace” → create workspace on first message of session.

5.8 UI

New panel or Git-adjacent tab **Workspaces**:

- List active workspaces with task/run link, branch, dirty badge
- Diff viewer (file list + unified patch; reuse git-panel patterns where possible)
- Actions: Merge / Discard / Open in shell (`cwd`) / Open chat session

Kanban card: show workspace badge + link when `workspace_id` present.

5.9 Tests

Test	Assert
create worktree	second worktree concurrent; primary branch unchanged
dirty isolation	agent A dirty files invisible in agent B tree
discard	path gone; branch deleted optional; primary clean
merge	feature commits on base; workspace status merged
non-git project	sandbox_copy mode or graceful skip with clear error
lock	parallel create does not corrupt worktree list

5.10 Acceptance criteria

- [] Two concurrent Kanban implement runs never checkout the main worktree
- [] UI can show live diff and discard safely
- [] Orphan worktrees detectable (`git worktree list` reconcile on boot)
- [] All worktree roots under allowed path policy

5.11 Migration / rollout

- Feature flag `CLOUDCLI_WORKSPACES=1` (default on for Kanban implement)
- Keep `feature_branch` column; also store `workspace_id` (new column on `kanban_tasks` or only on runs)
- Reconcile on server boot: mark missing dirs as orphan, offer cleanup

6. Phase 2 — Canonical Run Spine + Run Observatory v1

6.1 Problem

Execution state is fragmented: in-memory chat runs, thin `kanban_runs`, separate MC/webhook delivery records. No single place for tokens, model, duration, or timeline.

6.2 Goals

- First-class `agent_runs` row for **every** execution path.

- Append-only agent_run_events (or file-backed blob for large streams).
- Bridge existing systems without big-bang rewrite.
- UI: Run Observatory — timeline, filters, cost/token summary when available.

6.3 Data model

```
CREATE TABLE IF NOT EXISTS agent_runs (
  run_id          TEXT PRIMARY KEY NOT NULL,
  project_id      TEXT,
  source          TEXT NOT NULL, -- chat|kanban|mission_control|webhook|automation|system
  source_ref      TEXT,          -- task_id | section_id | delivery_id | session_id
  workspace_id    TEXT,
  app_session_id  TEXT,
  provider        TEXT,
  model           TEXT,
  effort          TEXT,
  permission_mode TEXT,
  profile_id      TEXT,
  status          TEXT NOT NULL DEFAULT 'queued',
  -- queued|starting|running|waiting_permission|waiting_approval|succeeded|failed|aborted|timed_out
  trigger         TEXT, --
  manual|schedule|webhook|dependency|column_move|failover|replay
  parent_run_id   TEXT,          -- failover/retry/fork
  root_run_id     TEXT,          -- group lineage
  title           TEXT,
  error_summary   TEXT,
  exit_code       INTEGER,
  token_input     INTEGER,
  token_output    INTEGER,
  token_total     INTEGER,
  cost_usd_estimate REAL,
  started_at      DATETIME,
  first_token_at  DATETIME,
  finished_at     DATETIME,
  created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
  meta_json       TEXT DEFAULT '{}')
);

CREATE INDEX IF NOT EXISTS idx_agent_runs_project_created ON agent_runs(project_id, created_at
DESC);
CREATE INDEX IF NOT EXISTS idx_agent_runs_status ON agent_runs(status);
CREATE INDEX IF NOT EXISTS idx_agent_runs_session ON agent_runs(app_session_id);
CREATE INDEX IF NOT EXISTS idx_agent_runs_source ON agent_runs(source, source_ref);

CREATE TABLE IF NOT EXISTS agent_run_events (
  event_id      TEXT PRIMARY KEY NOT NULL,
  run_id        TEXT NOT NULL,
  seq           INTEGER NOT NULL,
  ts            DATETIME NOT NULL,
  source        TEXT,
  type          TEXT NOT NULL,
  severity      TEXT DEFAULT 'info',
  payload_json  TEXT DEFAULT '{}',
  FOREIGN KEY (run_id) REFERENCES agent_runs(run_id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_agent_run_events_run_seq ON agent_run_events(run_id, seq);

-- Optional retention: payload blobs > N KB stored on disk under
-- tmp is wrong for durable; use ~/.cloudcli/runs/<run_id>/events/... or project .cloudcli/runs/
```

Bridge columns (additive):

- kanban_runs.run_id already exists → set equal to spine run_id going forward (or add agent_run_id FK). Prefer **unify IDs**: create spine row first, use same id in kanban_runs.
- Chat: on stream start, create spine run; registry holds pointer run_id.
- MC items / webhook deliveries: add agent_run_id nullable column.

6.4 Event types (v1 minimum)

type	when
run.queued	enqueued
run.started	process/session start
run.first_token	first model token
run.status	status transition
model.selected	provider/model/effort
workspace.bound	workspace_id
tool.call / tool.result	tool use (redact secrets)
permission.requested / permission.resolved	HITL
approval.requested / approval.resolved	MC
token.usage	usage snapshot
git.commit / git.diff_summary	shipping hooks
test.started / test.finished	ship loop
run.completed / run.failed / run.aborted	terminal
failover.triggered	P9
pack.attached	P7

6.5 Service API

```
interface RunService {
  create(input: CreateRunInput): AgentRun;
  get(runId: string): AgentRun | null;
  list(filter: RunListFilter): { runs: AgentRunSummary[]; nextCursor?: string };
  updateStatus(runId: string, status: RunStatus, patch?: Partial<AgentRun>): void;
  appendEvent(runId: string, event: Omit<RunEventEnvelope, 'event_id' | 'seq'>): RunEventEnvelope;
  listEvents(runId: string, opts?: { afterSeq?: number; limit?: number }): RunEventEnvelope[];
  attachUsage(runId: string, usage: TokenUsage): void;
  linkSession(runId: string, appSessionId: string): void;
  linkWorkspace(runId: string, workspaceId: string): void;
  markTerminal(runId: string, result: TerminalResult): void;
  /** Reconcile: running runs with dead processes → failed/aborted */
  reconcileOrphans(): number;
}
```

Retention policy v1:

- Keep run rows 90 days (configurable)
- Keep full events 14 days; then compact to summary JSON on run row
- Never store raw prompts with secrets; store refs/hashes if needed

6.6 Chat-run-registry migration

- Keep in-memory registry for live sockets.
- On create: also runService.create.
- On significant events: appendEvent (sample tool calls if volume high — v1 can cap tool payloads to 4KB).
- On complete: markTerminal.
- Remove “5 minute only” as sole source of truth (if TTL exists, TTL becomes cache, not record of truth).

6.7 HTTP + WS

```
GET /api/runs?projectId&status&source&from&to&cursor&limit
GET /api/runs/:runId
GET /api/runs/:runId/events?afterSeq&limit
POST /api/runs/:runId/abort
```

WS: run_event, run_updated.

6.8 UI — Run Observatory v1

- Global or per-project page: table of runs (status, provider, model, duration, tokens, source)
- Detail drawer: timeline of events, link to session, workspace, task
- Stuck detection: running && no event for N minutes (default 15) → badge + interrupt

6.9 Tests

- Create/list/filter runs
- Event seq monotonic
- Bridge: kanban finish updates spine
- Reconcile orphans on boot
- Redaction: payload with Authorization header stripped

6.10 Acceptance criteria

- [] New chat message creates durable run row
- [] Kanban run shares spine id
- [] Observatory shows timeline for last 24h
- [] Server restart does not lose completed run metadata

7. Phase 3 — Interrupt Queue (Attention Budget)

7.1 Problem

Approvals, permissions, failures, overdue cards, and auth health live in separate UIs. Notifications are largely passive. Remote/mobile users cannot clear “what needs me” quickly.

7.2 Goals

Single prioritized **Interrupt Queue** with actions: Approve, Deny, Resume, Open, Snooze, Dismiss, Delegate (v1: reassign provider/profile only).

7.3 Data model

```
CREATE TABLE IF NOT EXISTS interrupts (  
  interrupt_id TEXT PRIMARY KEY NOT NULL,  
  project_id TEXT,  
  kind TEXT NOT NULL,  
  -- permission_pending | approval_pending | run_failed | run_stuck  
  -- auth_unhealthy | mcp_unhealthy | task_overdue | task_blocked  
  -- workspace_conflict | secret_missing | ci_failed  
  severity TEXT NOT NULL DEFAULT 'warning', -- info|warning|error|critical  
  title TEXT NOT NULL,  
  body TEXT DEFAULT '',  
  run_id TEXT,  
  task_id TEXT,  
  workspace_id TEXT,  
  href TEXT,  
  actions_json TEXT NOT NULL DEFAULT '[]',  
  -- [{id,label,style,handler}] handler is server-known action key  
  status TEXT NOT NULL DEFAULT 'open', -- open|snoozed|resolved|dismissed  
  snooze_until DATETIME,  
  resolved_at DATETIME,  
  resolved_by TEXT,  
  resolution TEXT,  
  priority INTEGER NOT NULL DEFAULT 50, -- lower = higher priority  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  meta_json TEXT DEFAULT '{}'  
);
```

```
CREATE INDEX IF NOT EXISTS idx_interrupts_open ON interrupts(status, priority, created_at);
CREATE INDEX IF NOT EXISTS idx_interrupts_project ON interrupts(project_id, status);
```

Priority defaults:

kind	priority
permission_pending	10
approval_pending	15
auth_unhealthy	20
run_stuck	25
run_failed	30
ci_failed	35
mcp_unhealthy	40
workspace_conflict	45
task_overdue	50
task_blocked	55
secret_missing	20

7.4 Producers (wire-ups)

Source	Event	Interrupt
Chat permission banner	permission request	permission_pending
Mission Control	item needs review	approval_pending
Run spine	terminal failed	run_failed
Run spine	stuck detector	run_stuck
Auth health	unhealthy	auth_unhealthy
MCP health	unhealthy	mcp_unhealthy
Kanban scheduler	overdue	task_overdue
Ship loop (P6)	CI fail	ci_failed
Secrets (P4)	missing ref	secret_missing

Dedup key: (kind, run_id|task_id|provider|mcp_name) — upsert rather than spam.

7.5 Actions (server handlers)

```
type InterruptActionKey =
  | 'approve_permission'
  | 'deny_permission'
  | 'approve_mc_item'
  | 'deny_mc_item'
  | 'resume_run'
  | 'abort_run'
  | 'retry_run'
  | 'open_href'
  | 'snooze'
  | 'dismiss'
  | 'delegate_provider'; // body: { provider, profileId? }
```

Snooze presets: 15m, 1h, 4h, tomorrow 9:00 local (server stores UTC).

7.6 API

```
GET /api/interrupts?status=open&projectId&limit
GET /api/interrupts/count
POST /api/interrupts/:id/actions/:actionKey
POST /api/interrupts/:id/snooze { until | preset }
POST /api/interrupts/plan-my-day // optional v1.1: returns ordered checklist
```

7.7 UI

- Sidebar entry **Needs you** with badge count (replace or augment notifications)
- Mobile-first list: large tap targets, primary action visible
- Deep link into chat permission panel / MC item / run / task
- “Plan my day” (v1.1): sorts open interrupts + overdue tasks into morning checklist

7.8 Migration

- Map existing `system_notifications` → create interrupts for actionable kinds; keep notifications as log feed if desired
- Avoid double UI noise: actionable → interrupt; informational → notification

7.9 Acceptance criteria

- [] Permission request appears in queue within 1s of WS event
- [] Approve from queue resolves chat permission
- [] Snooze hides until time; returns automatically
- [] Badge count matches open non-snoozed interrupts

8. Phase 4 — Secrets Hygiene (Vault v1)

8.1 Problem

`user_credentials.credential_value` and webhook HMAC secrets are plaintext. MCP env may embed tokens. Logs risk leakage.

8.2 Goals

- Secret store with **OS keychain when available**, else encrypted-at-rest DB (libsodium/secretbox or safe equivalent already acceptable in Node ecosystem).
- Reference syntax: `${secret:NAME}` or `${secret:sec_...}`.
- Resolve at runtime for: credentials, MCP env, webhook secrets, stack capsule.
- Redact secrets in logs, run events, and error messages.

8.3 Data model

```
CREATE TABLE IF NOT EXISTS secrets (  
  secret_id      TEXT PRIMARY KEY NOT NULL,  
  name           TEXT NOT NULL,           -- unique per scope  
  scope          TEXT NOT NULL DEFAULT 'user', -- user|project|provider|profile  
  scope_ref      TEXT,                   -- project_id / provider / profile_id  
  backend        TEXT NOT NULL,          -- keychain|encrypted_db  
  keychain_account TEXT,                 -- when backend=keychain  
  ciphertext     BLOB,                   -- when encrypted_db  
  nonce          BLOB,  
  content_type   TEXT DEFAULT 'token',   -- token|json|file_ref  
  description    TEXT,  
  last_used_at   DATETIME,  
  expires_at     DATETIME,  
  created_at     DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at     DATETIME DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE(name, scope, scope_ref)  
);  
  
-- Migration: user_credentials.credential_value may become secret_id ref  
-- ALTER user_credentials ADD COLUMN secret_id TEXT;  
-- Keep credential_value nullable during migration; dual-read then drop later
```

Master key:

- Env `CLOUDCLI_SECRETS_KEY` (base64 32 bytes) or generated once into OS keychain / app_config **file mode 0600** — document threat model: local attacker with FS access may still win without OS keychain.

8.4 Resolve API

```
interface SecretsService {
  put(input: { name: string; value: string; scope?: Scope; description?: string }): SecretMeta;
  getMeta(secretIdOrName: string, scope?: Scope): SecretMeta | null;
  resolve(ref: string, ctx: ResolveContext): string; // throws SECRET_NOT_FOUND
  resolveInObject<T>(obj: T, ctx: ResolveContext): T; // deep replace ${secret:...}
  delete(secretId: string): void;
  list(scope?: Scope): SecretMeta[]; // never returns values
  redact(text: string): string; // replace known secret values / patterns
}
```

Ref grammar:

```
${secret:GITHUB_TOKEN}
${secret:sec_01JABC...}
${secret:project:myproj:NPM_TOKEN} // optional qualified form
```

8.5 Integration points

- Git credentials repository
- MCP catalog env projection
- Webhook source HMAC secret
- Provider env if any
- Run event serializer calls redact
- Plugin start env

8.6 UI

Settings → **Secrets**: list names, scopes, last used, expiry; add/rotate/delete; never show full value after save (show last 4 optional).

8.7 Tests

- Put/resolve/delete
- Deep resolve in MCP config JSON
- Redaction strips token from log line
- Migration dual-read credentials
- Missing secret → actionable interrupt

8.8 Acceptance criteria

- [] New credentials never written plaintext
- [] Existing rows migratable via one-shot command
- [] MCP env supports secret refs end-to-end
- [] Run events do not contain known secret values

9. Phase 5 — One Work System + PRD→Delivery Graph

9.1 Problem

TaskMaster (.taskmaster) and native Kanban coexist. PRD “Generate Tasks” is a stub modal telling users to ask Claude manually.

9.2 Goals

- **Kanban is the system of record** for execution.
- Import TaskMaster tasks → Kanban (one-way v1; optional sync later).
- PRD → editable **Delivery Graph preview** → create Kanban tasks + deps.

- Deprecate TaskMaster UI paths gradually (feature flag hide).

9.3 Delivery graph schema (preview, not yet persisted as separate product)

```
type DeliveryGraph = {
  version: 1;
  prdPath: string;
  title: string;
  requirements: Array<{ id: string; text: string; priority?: number }>;
  acceptanceCriteria: Array<{ id: string; text: string; reqIds?: string[] }>;
  tasks: Array<{
    tempId: string;
    title: string;
    description: string;
    prompt: string;
    reqIds: string[];
    acceptanceIds: string[];
    dependsOn: string[]; // tempIds
    estimateMinutes?: number;
    assigneeProvider?: string;
    reviewProvider?: string;
    implementProfileId?: string;
    reviewProfileId?: string;
    permissionMode?: string;
    suggestedBranch?: string;
    labels?: string[];
  }>;
  schedule?: { start?: string; strategy?: 'asap' | 'sequential' };
  mcps?: string[]; // suggested catalog names
  skills?: string[];
};
```

9.4 Generation pipeline

- Read PRD file from project (TaskMaster docs path **or** arbitrary path).
- Build generator prompt with schema above (strict JSON).
- Run via provider session (user-selected provider/model) **or** lightweight structured generate already used by `kanban-generate.service.ts`.
- Return graph to UI for edit.
- On approve:
 - create board if needed
 - create tasks with deps (`kanban_task_deps`)
 - optional: enqueue ready tasks (no deps) if `startReady: true`
 - each implement task uses workspaces (P1)

9.5 API

```
POST /api/projects/:projectId/delivery-graph/generate
{ prdPath, provider?, model?, boardId? }
POST /api/projects/:projectId/delivery-graph/apply
{ graph, boardId, startReady?: boolean }
POST /api/projects/:projectId/taskmaster/import
{ boardId, path?: '.taskmaster/tasks/tasks.json' }
```

9.6 UI

- Replace `GenerateTasksModal` stub with multi-step: Generate → Edit graph (list + dep visualization simple) → Apply
- TaskMaster panel: “Import to Kanban” primary CTA; badge “Legacy”

9.7 Acceptance criteria

- [] PRD generate produces editable tasks without manual chat
- [] Deps preserved in Kanban
- [] TaskMaster import maps status/columns reasonably

- [] No data loss on import (dry-run report)

10. Phase 6 — Ship Loop (Diff → Test → PR → CI)

10.1 Problem

Agent finishes in a workspace; human still manually does test/PR/CI outside a unified flow.

10.2 Goals

From a run/workspace:

- Review diff (file + optional hunk accept/reject v1.1; v1 file-level)
- Run test command (project-configured or auto-detect)
- Create PR (gh/gitlab)
- Show CI status
- “Fix failing check” creates follow-up run (child parent_run_id)

10.3 Config

```
# .cloudcli/ship.yaml (or section of stack.yaml in P10)
test:
  command: "npm test"
  cwd: "."
pr:
  baseBranch: "main"
  draft: true
  reviewers: []
ci:
  provider: "github" # github|gitlab|none
  pollSeconds: 30
```

10.4 Service

```
interface ShipService {
  runTests(workspaceId: string): Promise<TestReport>;
  createPullRequest(workspaceId: string, input: PrInput): Promise<PullRequest>;
  getCiStatus(prUrlOrId: string): Promise<CiStatus>;
  openFixRun(input: { parentRunId: string; failureSummary: string }): Promise<AgentRun>;
}
```

Use existing git routes where possible; extract to server/modules/git/ if git.js is too large.

10.5 API / UI

```
POST /api/workspaces/:id/ship/test
POST /api/workspaces/:id/ship/pr
GET /api/workspaces/:id/ship/ci
POST /api/runs/:runId/ship/fix-ci
```

UI: **Ship** stepper on workspace detail (Diff → Test → PR → CI).

10.6 Acceptance criteria

- [] PR created from workspace branch with test status comment
- [] CI failure creates interrupt + optional fix run
- [] Works with GitHub gh when authenticated via secret ref

11. Phase 7 — Context Pack Compiler

11.1 Problem

Agents either get too little context or dump entire repos. Semantic “brain” is large; handoff/memory already exist.

11.2 Goals

Given a task or free-text goal + token budget, compile a **Context Pack**:

- Ranked file paths + excerpts
- Memory notes (Obsidian) hits
- Prior run summaries
- Open Kanban deps / comments
- Git blame/hot files optional

Output: markdown pack + machine JSON; attach to session/run.

11.3 Types

```
type ContextPack = {
  pack_id: string;
  project_id: string;
  goal: string;
  budgetTokens: number;
  estimatedTokens: number;
  items: Array<{
    kind: 'file' | 'memory' | 'run_summary' | 'task' | 'diff' | 'adr';
    uri: string;
    title: string;
    excerpt: string;
    score: number;
    freshAt?: string;
  }>;
  warnings: string[]; // conflicts, stale, budget trimmed
  created_at: string;
};
```

11.4 Algorithm (v1 — no vector DB required)

- Parse goal + task description keywords
- Candidate files:
- paths mentioned in task
- rg/git grep top hits
- recently changed files (git log --since)
- Memory: Obsidian search if configured
- Runs: last N succeeded/failed summaries from spine for same project
- Score = lexical overlap + recency + user pin
- Greedy fill until budget
- Emit pack; store under ~/.cloudcli/packs/ or DB table context_packs

11.5 API

```
POST /api/projects/:projectId/context-packs
  { goal, taskId?, budgetTokens?, runId? }
GET /api/context-packs/:packId
POST /api/runs/:runId/attach-pack { packId }
POST /api/sessions/:sessionId/attach-pack { packId }
```

11.6 UI

- Button on Kanban task / chat composer: **Build context pack**
- Preview items with include/exclude toggles
- Attach → injects as system/handoff section

11.7 Acceptance criteria

- [] Pack stays under budget $\pm 10\%$
- [] Citations (uris) present for every item

- [] Attach visible in run events (pack.attached)

12. Phase 8 — Shared Automation Kernel

12.1 Problem

Kanban schedules, Mission Control sections, and Webhooks each implement trigger/run/retry differently.

12.2 Goals

One kernel:

trigger → conditions → context build → action(s) → post-hooks

Actions: start_agent_run, enqueue_kanban_task, http_webhook_out, notify, create_interrupt, noop.

v1 is **API + JSON recipes**, not a full visual studio.

12.3 Data model

```
CREATE TABLE IF NOT EXISTS automation_recipes (  
  recipe_id      TEXT PRIMARY KEY,  
  name           TEXT NOT NULL,  
  enabled        INTEGER NOT NULL DEFAULT 1,  
  version        INTEGER NOT NULL DEFAULT 1,  
  project_id     TEXT,                -- null = global  
  trigger_json   TEXT NOT NULL,  
  conditions_json TEXT DEFAULT '[]',  
  actions_json   TEXT NOT NULL,  
  retry_json     TEXT DEFAULT '{"max":0}',  
  timeout_ms     INTEGER,  
  created_at     DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at     DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE IF NOT EXISTS automation_runs (  
  automation_run_id TEXT PRIMARY KEY,  
  recipe_id         TEXT NOT NULL,  
  agent_run_id      TEXT,            -- spine link  
  status            TEXT,  
  attempt           INTEGER DEFAULT 1,  
  trigger_payload_json TEXT,  
  error             TEXT,  
  started_at        DATETIME,  
  finished_at       DATETIME  
);
```

Trigger types: cron, webhook_inbound, kanban_event, run_completed, interrupt_created, git_event (v1.1), manual.

12.4 Migration strategy

Legacy	Approach
Kanban schedule_cron	Compile to recipe behind the scenes OR dual-run with flag
MC sections	Runner calls kernel start_agent_run action
Webhooks	Ingest → kernel trigger webhook_inbound

Do not break existing tables in v1; **adapter layer** wraps kernel.

12.5 Acceptance criteria

- [] One recipe can fire on cron and produce spine run
- [] Webhook delivery path uses kernel without behavior regression
- [] Retries honor retry_json
- [] Cycle detection on kanban_event → action that re-emits same event

13. Phase 9 — Provider Failover Playbooks

13.1 Problem

Auth/MCP health exists; when Claude dies mid-work, user must manually handoff.

13.2 Goals

Declarative playbooks:

```
type FailoverPlaybook = {
  playbook_id: string;
  name: string;
  match: {
    providers?: string[];
    errors?: Array<'auth' | 'rate_limit' | 'timeout' | 'mcp_unhealthy' | 'any'>;
  };
  strategy: {
    candidates: Array<{ provider: string; model?: string; profileId?: string }>;
    handoffMode: 'summary' | 'full' | 'fresh';
    attachContextPack?: boolean;
    maxFailovers: number;
  };
  approval: 'auto' | 'interrupt'; // interrupt creates P3 item
};
```

13.3 Flow

- Detect failure class from run terminal error / auth-health
- Match playbook
- If approval: interrupt → create interrupt with action approve_failover
- On auto/approve: build handoff (existing service) + optional context pack
- runService.create with parent_run_id, trigger: failover
- Start candidate provider

13.4 API / UI

```
GET/POST /api/failover-playbooks
POST /api/runs/:runId/failover { playbookId? }
```

Settings: ordered candidate list per primary provider.

13.5 Acceptance criteria

- [] Simulated auth failure triggers playbook
- [] Child run linked via parent_run_id
- [] No infinite failover loops (maxFailovers)

14. Phase 10 — Workspace Capsule + Doctor

14.1 Goals

Repo-owned .cloudcli/stack.yaml describing:

- required providers
- MCP bindings (catalog names + secret refs)
- skills
- agent profiles (by name)
- memory config
- ship config
- notification preferences (non-secret)
- health expectations

Commands/API: doctor, plan, apply, export, restore.

14.2 stack.yaml example

```
version: 1
project: cloudcli-fork
providers:
  required: [claude, grok]
  optional: [codex, cursor]
mcp:
  - name: obsidian
    enabledFor: [claude, grok]
    env:
      OBSIDIAN_API_KEY: ${secret:OBSIDIAN_API_KEY}
skills:
  global: [project-memory]
  project: []
memory:
  kind: obsidian
  vaultRelative: null
ship:
  test:
    command: npm test
health:
  auth: [claude, grok]
  mcp: [obsidian]
```

14.3 Doctor checks

Check	Fail criteria
provider binary	missing / version
auth	auth-health unhealthy
mcp	catalog missing / health fail
secrets	unresolved refs
worktrees	orphans
gitignore	.cloudcli/worktrees missing
skills	fan-out drift

Exit code non-zero on fail; JSON report for UI.

14.4 API

```
GET  /api/projects/:id/stack
PUT  /api/projects/:id/stack
POST /api/projects/:id/stack/doctor
POST /api/projects/:id/stack/apply
POST /api/projects/:id/stack/export
```

14.5 Acceptance criteria

- [] Fresh machine: apply + secrets refs → doctor green (given keys)
- [] Export does not include secret values
- [] Doctor surfaces actionable interrupts for failures

15. Phase 11 — Phase-2 platforms (backlog PRDs)

Implement only after P1–P2 are solid. Each is a separate future PRD; summary contracts only.

15.1 Replayable Evals & Time Machine

- Replay manifest: prompt, commit SHA, workspace snapshot ref, provider/model, MCP versions, permissions, pack id, outputs, tests

- POST /api/evals/from-run/:runId
- Compare matrix across providers

15.2 Multi-Agent Review Swarm

- Run group with roles: planner, implementer, tester, security, docs
- Shared artifact store; final approval gate
- Requires workspaces + spine

15.3 Self-registering Provider SDK

- Signed manifest: runtime, auth, models, capabilities, session paths
- Conformance test suite
- Dynamic registry load

15.4 Zero-trust Supply Chain (beyond vault)

- Plugin signature verify
- MCP tool scopes (fs/network)
- Immutable audit log
- Drift scanning

15.5 Semantic Workspace Brain

- Index sessions (all providers), commits, kanban comments, MC outputs, memory
- Hybrid retrieval (BM25 + embeddings optional)
- Feeds Context Pack Compiler as candidate source

15.6 Outcome-driven Model Router

- Durable outcome metrics from spine
- Policies: fastest, cheapest, best_debug, private_local
- Trained thresholds from user ratings

15.7 Browser Takeover & Evidence

- Human takeover of agent browser session
- HAR, Playwright traces, screenshots attached to run

15.8 Visual Automation Studio

- UI on top of P8 kernel
- Versioned recipes, typed outputs, command-palette macros

16. Cross-cutting architecture

16.1 Target module map

```
server/modules/
workspaces/      # P1
runs/            # P2
interrupt-queue/ # P3
secrets/         # P4
delivery-graph/  # P5
ship/            # P6
context-pack/    # P7
automation/      # P8
failover/        # P9
```

```

stack/           # P10
kanban/          # existing – adapters
mission-control/ # existing – adapters
webhooks/        # existing – adapters
providers/       # existing – failover + cwd
auth-health/     # existing – producers
database/        # migrations

```

16.2 Migration discipline

- Additive SQL only in `server/modules/database/migrations.ts / schema.ts`
- Feature flags in `app_config` or `env`:
- `FEATURE_WORKSPACES`
- `FEATURE_RUN_SPINE`
- `FEATURE_INTERRUPT_QUEUE`
- `FEATURE_SECRETS_VAULT`
- `FEATURE_DELIVERY_GRAPH`
- `FEATURE_SHIP_LOOP`
- `FEATURE_CONTEXT_PACK`
- `FEATURE_AUTOMATION_KERNEL`
- `FEATURE_FAILOVER`
- `FEATURE_STACK_CAPSULE`

16.3 Testing standards

Layer	Tooling
Unit	existing <code>node:test / vitest</code> patterns in repo
Module integration	temp DB under <code>tmp/cloudcli/test-<id>/</code>
Git worktree	real git init in tmp
UI	light component tests if present; else manual checklist in PR

16.4 Security checklist (every phase)

- ☐ No secret in events/logs
- ☐ Path traversal blocked on workspace roots
- ☐ SSRF care on webhooks
- ☐ Command injection care on ship test commands (allowlist or `shell:false + argv`)
- ☐ Plugin code not auto-escalated

16.5 Observability

- Prefer spine events over ad-hoc `console.log` for user-visible state
- Optional future: OpenTelemetry GenAI export (Phase 2)

17. Multi-agent implementation playbooks

17.1 Suggested sprint slices (2–4 agent waves)

Wave A (foundations, parallel):

- Agent A1: P0 types + P2 schema/repository only
- Agent A2: P1 workspace service + tests (no kanban wire)
- Agent A3: P4 secrets service + tests

Wave B:

- Agent B1: P1 kanban/runner wiring + flag
- Agent B2: P2 chat/kanban bridge + events
- Agent B3: P4 migration + MCP resolve

Wave C:

- Agent C1: P1 + P2 UI (workspaces + observatory)
- Agent C2: P3 interrupt queue backend + producers
- Agent C3: P3 UI

Wave D:

- Agent D1: P5 delivery graph
- Agent D2: P6 ship loop
- Agent D3: P7 context pack

Wave E:

- Agent E1: P8 kernel + webhook adapter
- Agent E2: P8 MC + kanban schedule adapters
- Agent E3: P9 failover
- Agent E4: P10 stack capsule + doctor

17.2 File ownership to avoid merge hell

Path	Primary owner wave
server/modules/database/schema.ts	serialize merges — one agent at a time or stacked PRs
server/modules/kanban/kanban-runner.service.ts	P1 then P2
server/modules/websocket/**	P2 then P3
src/components/app/AppContent.tsx/sidebar	P3 (nav) — coordinate
server/routes/git.js	P6 (or extract module first)

Rule: Schema changes go through a single “schema steward” PR per wave.

17.3 Agent prompt template

You are implementing CloudCLI PRD phase <Pn> slice <WS-...>.
Read: docs/prd/CloudCLI-Feature-PRD-v1.md section <...>
Constraints: tmp/cloudcli only; no plaintext secrets; additive migrations;
do not expand scope to other phases.
Deliver: code + tests + brief summary of APIs added.

18. Success metrics

Metric	Baseline (today)	Target (after P1–P3)
Concurrent agent worktree conflicts	possible	0 on Kanban implement
Time to find “what needs me”	multi-panel	< 2s open queue
Durable run after restart	chat lost	100% metadata retained
Plaintext credentials	yes	0 new; migrate 100%
PRD to tasks	manual chat	1-click generate+apply
Failover after auth fail	manual	1-click or auto playbook

19. Open questions (resolve during P0/P1)

- ULID library vs UUID v7 vs existing UUID — pick one.
- Worktree root: always under project .cloudcli/ vs global ~/.cloudcli/worktrees? (**Recommend project-local + gitignore.**)
- SQLite vs file storage for large run event payloads? (**Recommend SQLite with size cap + external blob path.**)
- TaskMaster: hard deprecate timeline? (**Recommend 2 releases with import + hide flag.**)
- Keychain package on Linux/macOS/Windows matrix for Electron?

20. Appendix A — Column additions cheat sheet

```
-- kanban_tasks
ALTER TABLE kanban_tasks ADD COLUMN workspace_id TEXT;

-- kanban_runs: prefer agent_runs.run_id == kanban_runs.run_id
-- else:
ALTER TABLE kanban_runs ADD COLUMN agent_run_id TEXT;

-- mc_items / webhook_deliveries
ALTER TABLE mc_items ADD COLUMN agent_run_id TEXT;
ALTER TABLE webhook_deliveries ADD COLUMN agent_run_id TEXT;

-- user_credentials
ALTER TABLE user_credentials ADD COLUMN secret_id TEXT;

-- sessions (if table allows meta)
-- store workspace_id in existing meta or new column
```

21. Appendix B — Minimal REST surface checklist

Phase	Methods
P1	CRUD-ish workspaces + diff/merge/discard
P2	list/get runs + events + abort
P3	list interrupts + actions + snooze
P4	secrets meta CRUD + resolve internal only
P5	generate/apply graph + taskmaster import
P6	test/pr/ci/fix
P7	create/get/attach pack
P8	recipes CRUD + manual trigger
P9	playbooks CRUD + failover
P10	stack get/put/doctor/apply/export

22. Appendix C — Related existing files (start here)

```
server/modules/kanban/git-branch.service.ts
server/modules/kanban/kanban-runner.service.ts
server/modules/kanban/kanban-queue.service.ts
server/modules/kanban/kanban.types.ts
server/modules/database/schema.ts
server/modules/database/migrations.ts
server/modules/websocket/services/chat-run-registry.service.ts
server/modules/providers/services/session-handoff.service.ts
server/modules/providers/services/provider-capabilities.service.ts
server/modules/auth-health/
server/modules/providers/services/session-conversations-search.service.ts
server/modules/providers/services/project-memory.service.ts
src/components/prd-editor/view/GenerateTasksModal.tsx
src/components/task-master/
```

src/components/git-panel/
src/components/sidebar/view/subcomponents/NotificationsPanel.tsx
server/routes/plugins.js
server/routes/git.js

23. Document history

Version	Date	Notes
1.0	2026-08-06	Initial master PRD from Sol/Luna/Grok synthesis; implementation-ready

End of PRD

End of Master Feature PRD v1.0 — implement from companion Markdown for full fidelity; this PDF is the portable reference for humans and review.